System F and Polymorphism

Lecture 11

Section 1

Recap

Where we left off

STLC computes only extended polynomials (Schwichtenberg). No exponentiation, no recursion.

Gödel's fix: **System T**. Add Nat, 0, S, and a typed recursor $\mathsf{R}_C : C \to (\mathsf{Nat} \to C \to C) \to \mathsf{Nat} \to C$.

Gödel (1958)

System T captures exactly the provably total functions of PA — far beyond primitive recursion (e.g. Ackermann).

But System T **left the pure lambda calculus** (added constructors, pattern matching) and its connection to PA is **indirect** (via the Dialectica interpretation, not Curry–Howard).

Today's question

Diagnosis	**Fix**	**Status**
No data or recursion	System T: add Nat + R	done
No generality over types	???	today

Can we increase computational power
by enriching the **logic**
rather than adding computational primitives?

Section 2

System F

The idea

STLC = propositional logic. No quantifiers.

The natural enrichment: **quantify over types**.

$\forall\alpha.\, \alpha \to \alpha$ as a proposition: "for all propositions α, α implies α."

A proof: a function that works at *every* type.

$$\Lambda\alpha.\, \lambda x{:}\alpha.\, x \quad : \quad \forall\alpha.\, \alpha \to \alpha$$

The idea

STLC = propositional logic. No quantifiers.

The natural enrichment: **quantify over types**.

$\forall\alpha.\, \alpha \to \alpha$ as a proposition: "for all propositions α, α implies α."

A proof: a function that works at *every* type.

$$\Lambda\alpha.\, \lambda x{:}\alpha.\, x \quad : \quad \forall\alpha.\, \alpha \to \alpha$$

Two new operations, both instances of abstraction/application:

- **Type abstraction**: $\Lambda\alpha.\, t$ and **type application**: $t\,[\![A]\!]$

New reduction: $(\Lambda\alpha.\, t)\,[\![A]\!] \longrightarrow_\beta t[\alpha := A]$

β-reduction at the type level, exactly parallel to $(\lambda x.\, t)\, s \to_\beta t[x := s]$.

Typing rules

Types: $A, B \;::=\; \alpha \mid A \to B \mid \forall \alpha.\, A$

Terms: $t, s \;::=\; x \mid \lambda x{:}A.\, t \mid t\ s \mid \Lambda \alpha.\, t \mid t\,[\![A]\!]$

$$\frac{\Gamma \vdash t : A \qquad \alpha \text{ not free in any type in } \Gamma}{\Gamma \vdash \Lambda \alpha.\, t \;:\; \forall \alpha.\, A} \quad (\forall\text{-I})$$

$$\frac{\Gamma \vdash t : \forall \alpha.\, A}{\Gamma \vdash t\,[\![B]\!] \;:\; A[\alpha := B]} \quad (\forall\text{-E})$$

Why the side condition? Without it, from $x{:}\alpha \vdash x : \alpha$ we could conclude $x{:}\alpha \vdash \Lambda \alpha.\, x : \forall \alpha.\, \alpha$. But $\forall \alpha.\, \alpha$ is the empty type $\bot$ — we would derive contradiction from any assumption. The side condition prevents quantifying over a variable our assumptions depend on.

What is second-order propositional logic?

Propositional logic: atomic propositions $p, q, r, \ldots$ combined with $\to$, $\wedge$, $\vee$, $\bot$. The connectives are **primitive**: each has its own rules.

Second-order: add $\forall\alpha.\, A$, where α ranges over *propositions*. We can now say "for all propositions α, ..."

Intuitionistic: constructive (no law of excluded middle). Same as STLC.

A key fact: in second-order propositional logic, $\wedge$, $\vee$, $\bot$, and $\exists$ are **definable** from $\forall$ and $\to$ alone:

$$\begin{aligned}
\bot &\equiv \forall\alpha.\, \alpha \\
A \wedge B &\equiv \forall\alpha.\, (A \to B \to \alpha) \to \alpha \\
A \vee B &\equiv \forall\alpha.\, (A \to \alpha) \to (B \to \alpha) \to \alpha \\
\exists\alpha.\, A &\equiv \forall\beta.\, (\forall\alpha.\, A \to \beta) \to \beta
\end{aligned}$$

Why the encodings work: $\bot$ and $\wedge$

Each encoding says: "to produce *any* result α, it suffices to…''

$\bot \equiv \forall\alpha.\,\alpha$: "I can produce any α you ask for, from nothing.'' No function can do this — there is no way to manufacture a value of an arbitrary unknown type. So $\bot$ has no proof.

$A \wedge B \equiv \forall\alpha.\,(A \to B \to \alpha) \to \alpha$: "Give me any way to combine an A and a B into an α, and I'll produce that α for you.'' This is what it means to hold a pair: you have the two components, and you can feed them to any function.

pair $a\ b = \Lambda\alpha.\,\lambda f{:}A \to B \to \alpha.\ f\ a\ b$ $\quad\{$(store a, b; apply any f)$\}$

fst $p = p\,[\![A]\!]\,(\lambda a{:}A.\,\lambda b{:}B.\ a)$ $\quad\{$(use f that keeps the first)$\}$

Check:
fst (pair $a\ b$) $= (\Lambda\alpha.\,\lambda f.\ f\ a\ b)\,[\![A]\!]\,(\lambda a.\lambda b.\ a) \to_\beta (\lambda f.\ f\ a\ b)\,(\lambda a.\lambda b.\ a) \to_\beta a.$

Why the encodings work: $\vee$

$A \vee B \equiv \forall\alpha.\,(A \to \alpha) \to (B \to \alpha) \to \alpha$: "Give me a handler for the A case and a handler for the B case, and I'll run the right one.'' This is case analysis.

$\mathsf{inl}\ a = \Lambda\alpha.\,\lambda f{:}A \to \alpha.\,\lambda g{:}B \to \alpha.\ f\ a$ $\quad$ {(left injection: use the f handler)}

$\mathsf{inr}\ b = \Lambda\alpha.\,\lambda f{:}A \to \alpha.\,\lambda g{:}B \to \alpha.\ g\ b$ $\quad$ {(right injection: use the g handler)}

Check: inl ignores g entirely and just applies f to a. The caller provides both handlers, not knowing which one will be used — exactly how disjunction works.

Why the encodings work: $\exists$

$$\exists\alpha.\, A \;\equiv\; \forall\beta.\, (\forall\alpha.\, A \to \beta) \to \beta$$

Read this type carefully. The **producer** (the one who inhabits this type) must:

- Accept any result type β (the outer $\forall\beta$)
- Accept a **consumer** $k : \forall\alpha.\, A \to \beta$ that works for *all* choices of α
- Produce a β

The producer has a *specific* type C in mind (the witness) and a proof $t : A[\alpha := C]$. Read this as: "C is my example of α, and t is my evidence that A holds for this choice.''

$$\text{pack}\,[C, t] \;=\; \Lambda\beta.\, \lambda k{:}(\forall\alpha.\, A \to \beta).\, k\,[\![C]\!]\,t$$

The key asymmetry: the *producer* chooses C, but the *consumer* k must handle *all possible* α. So k cannot inspect which type was packed. This is exactly "there exists an α, but I won't say which.''

The Curry–Howard correspondence

Claim: erasing term annotations from a System F typing derivation yields a proof in second-order intuitionistic propositional logic. A derivation **is** a proof.

Example: proof of $\forall\alpha.\ \alpha \to \alpha$:

Typing	**Rule**	**Logic (erase terms)**
$x : \alpha \vdash x : \alpha$	var	$\alpha \vdash \alpha$
$\vdash \lambda x{:}\alpha.\, x : \alpha \to \alpha$	$\to$-I	$\vdash \alpha \to \alpha$
$\vdash \Lambda\alpha.\, \lambda x{:}\alpha.\, x : \forall\alpha.\, \alpha \to \alpha$	$\forall$-I	$\vdash \forall\alpha.\, \alpha \to \alpha$

This works in general: every typing rule of System F *is* a logical rule (with proof terms attached). Conversely, every proof in second-order intuitionistic propositional logic can be decorated with terms to give a typing derivation. The connective definitions on the previous slide let us *derive* their intro/elim rules within the system.

Section 3

Recovering System T

Can we recover System T?

System T added Nat, 0, S, and a recursor R as **primitives**. Can System F **define** them using only Λ and $[\![\cdot]\!]$?

Church numerals:

$$\mathsf{Nat} \equiv \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha$$

"For any type α, given a step function $f : \alpha \to \alpha$ and a starting point $x : \alpha$, produce an α.'' The numeral $\overline{n}$ iterates f exactly n times.

$$\overline{0} = \Lambda\alpha.\,\lambda f{:}\alpha \to \alpha.\,\lambda x{:}\alpha.\,x$$
$$\overline{n} = \Lambda\alpha.\,\lambda f{:}\alpha \to \alpha.\,\lambda x{:}\alpha.\,f^n\,x$$
$$\mathsf{S} = \lambda n{:}\mathsf{Nat}.\,\Lambda\alpha.\,\lambda f{:}\alpha \to \alpha.\,\lambda x{:}\alpha.\,f\,(n\,[\![\alpha]\!]\,f\,x)$$

What the recursor does and why

Church numerals give **iteration**: $\overline{n}\,[\![C]\!]\;f\;x = f^n(x)$. The step function $f : C \to C$ sees only the accumulated value.

But many definitions need to know *which* step they're on. In Lean 4:

```
Nat.rec (motive := fun _ => C) (c : C) (h : Nat -> C -> C)
                        (n : Nat) : C
```

This is System T's recursor $\mathsf{R}_C : C \to (\mathsf{Nat} \to C \to C) \to \mathsf{Nat} \to C$:

$$\mathsf{R}\;c\;h\;\overline{0} = c \qquad \text{(base case: return } c\text{)}$$

$$\mathsf{R}\;c\;h\;(\mathsf{S}\;k) = h\;k\;(\mathsf{R}\;c\;h\;k) \qquad \text{(step: } h \text{ gets index } k \text{ and prev. result)}$$

Example: factorial. $c = 1$, $h = \lambda k.\,\lambda v.\,(k{+}1) \times v$. Then $\mathsf{R}\;1\;h\;3 = 3 \times 2 \times 1 = 6$.

Problem: Church numerals only iterate. Can we recover R?

Building the recursor from iteration

Idea: iterate on *pairs* $(\text{index},\ \text{accumulated value})$.

Define a step function on pairs:

$$\text{step} \;=\; \lambda p{:}\text{Nat} \times C.\ \text{pair}\ (\text{S}(\text{fst}\ p))\ (h\ (\text{fst}\ p)\ (\text{snd}\ p))$$

This maps $(k,\ v)$ to $(k{+}1,\ h\ k\ v)$. Each step increments the index and applies h with both the index and the current value — exactly what R needs.

Now iterate step starting from $(\overline{0},\ c)$:

$$\text{R}\ c\ h\ n \;=\; \text{snd}\Big(\overline{n}\ [\![\text{Nat} \times C]\!]\ \text{step}\ (\text{pair}\ \overline{0}\ c)\Big)$$

After n iterations of step, the first component is $\overline{n}$ and the second is $\text{R}\ c\ h\ n$. We extract the second component with snd.

The pairing trick in action

Trace R $c\ h\ \overline{2}$: iterate step twice from $(\overline{0}, c)$.

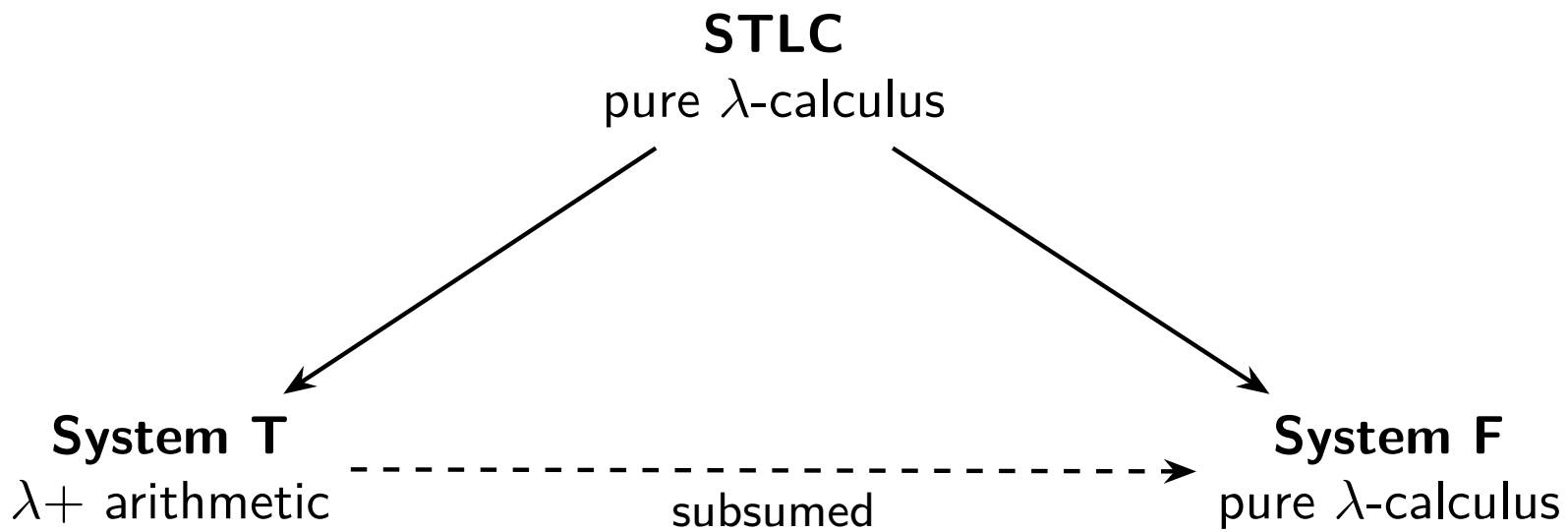
After	Pair	What happened
0 steps	$(\overline{0},\ c)$	base case
1 step	$(\overline{1},\ h\ \overline{0}\ c)$	step: index $0 \to 1$, applied $h\ \overline{0}\ c$
2 steps	$(\overline{2},\ h\ \overline{1}\ (h\ \overline{0}\ c))$	step: index $1 \to 2$, applied $h\ \overline{1}$ to prev.

snd of final pair: $h\ \overline{1}\ (h\ \overline{0}\ c) = \mathsf{R}\ c\ h\ \overline{2}$.

Pure iteration only sees the accumulated value. By pairing in the index, we give h the information it needs. This is what converts **iteration into recursion**.

System F subsumes System T

	System T	**System F**
Nat	primitive type	$\forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha$
0, S	primitive terms	Church-encodable
iteration	primitive combinator R	pairing trick (prev. slides)



No new data types. No pattern matching. Just abstraction/application lifted to the type level.

Section 4

Metatheory

Reduction and normal forms in System F

System F has two β-rules, both in **terms**:

$$(\lambda x{:}A.\, t)\ s \ \to_\beta\ t[x := s] \qquad\qquad (\Lambda\alpha.\, t)\ [\![B]\!] \ \to_\beta\ t[\alpha := B]$$

The second rule *substitutes a type for a type variable inside a term*. It does not "run'' anything at the type level: B is spliced in as-is, no further evaluation happens to it. Types have no reduction rules of their own.

A **normal form**: a term containing no subterm of either shape above.

The typing rules, for reference

Five rules. Every well-typed term is built from exactly these:

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}\ (\text{Var}) \qquad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x{:}A.\, t : A \to B}\ (\to\text{-I})$$

$$\frac{\Gamma \vdash t : A \to B \qquad \Gamma \vdash s : A}{\Gamma \vdash t\ s : B}\ (\to\text{-E})$$

$$\frac{\Gamma \vdash t : A \qquad \alpha \text{ not free in } \Gamma}{\Gamma \vdash \Lambda\alpha.\, t : \forall\alpha.\, A}\ (\forall\text{-I}) \qquad \frac{\Gamma \vdash t : \forall\alpha.\, A}{\Gamma \vdash t\,[\![B]\!] : A[\alpha := B]}\ (\forall\text{-E})$$

Rule inversion: if $\Gamma \vdash t : A$, then exactly one of these five rules was used last. The term t and type A tell us which.

Why $\bot = \forall\alpha.\,\alpha$ is uninhabited

Goal: no closed normal form has type $\forall\alpha.\,\alpha$.

Step 1 — invert $\vdash t : \forall\alpha.\,\alpha$. The type is $\forall\alpha.\,(\ldots)$, so the last rule must be $\forall$-**I**. Therefore $t = \Lambda\alpha.\,s$ and $\vdash s : \alpha$ in the empty context.

Step 2 — invert $\vdash s : \alpha$ (empty context, α is a type *variable*). Check each rule:

- **Var**: $s = x$ with $x : \alpha \in \Gamma$. But Γ is empty. ✗
- $\to$-**I**: $s = \lambda x{:}B.\,r$, giving type $B \to C$. But α is a variable, not $B \to C$. ✗
- $\forall$-**I**: $s = \Lambda\beta.\,r$, giving type $\forall\beta.\,D$. But α is a variable, not $\forall\beta.\,D$. ✗
- $\to$-**E**: $s = f\ u$ where $f : D \to \alpha$. In normal form, f must ultimately be headed by a variable. But Γ is empty: no variable is available. ✗
- $\forall$-**E**: $s = f\,[\![B]\!]$ where $f : \forall\beta.\,D$. Same problem: f needs a variable head, but none exist. ✗

All five rules fail. No closed normal form of type $\forall\alpha.\,\alpha$ exists. □

Strong normalization and consistency

Theorem (Girard, 1972)

Every well-typed System F term is strongly normalizing.

How this gives consistency (three ingredients):

1. **SN**: every well-typed term reduces to a normal form.
2. **Subject reduction**: if $\Gamma \vdash t : A$ and $t \to_\beta t'$, then $\Gamma \vdash t' : A$. (Type is preserved under reduction.)
3. **Previous slide**: no closed normal form has type $\forall\alpha.\,\alpha$.

Suppose for contradiction there exists a closed t with $\vdash t : \forall\alpha.\,\alpha$. By (1), t reduces to a normal form t^*. By (2), $\vdash t^* : \forall\alpha.\,\alpha$. But (3) says no such t^* exists. Contradiction. So $\forall\alpha.\,\alpha = \bot$ is uninhabited. (Proof of SN next week.)

PA_2 and the computational characterization

What is PA_2? In PA (Peano Arithmetic), induction is an axiom *schema*: one instance per formula. **PA_2 (second-order arithmetic)** adds variables ranging over *subsets* of $\mathbb{N}$ (equivalently, properties of naturals) and quantifies over them with $\forall X$.

Connection to System F: in PA_2, $\forall X$ ranges over properties of naturals. In System F, $\forall \alpha$ ranges over types. Under Curry–Howard, types *are* propositions, and propositions about naturals *are* properties — so type variables play the role of set variables.

Computational characterization

System F represents exactly the **provably total functions of PA_2**.

PA_2 proves Con(PA), which PA itself cannot. So System F computes strictly more than System T.

Section 5

The Price of Power

What System F loses

In STLC, three properties hold:

1. **Type checking**: given t and A, decide whether $\Gamma \vdash t : A$.
2. **Inhabitation**: given A, decide whether any closed $t : A$ exists.
3. **Completeness**: every propositional tautology is provable, hence inhabited.

System F **keeps** (1): type checking is decidable because the term t carries all the information. At each $\Lambda\alpha$, the type variable is bound; at each $t\,[\![B]\!]$, the type B is explicitly written. We just verify each rule bottom-up — no search is needed.

System F **loses** (2) and (3). We explain why next.

Why normalization doesn't help

Tempting reasoning: System F is strongly normalizing, so every well-typed term has a normal form, so we can search over normal forms, so inhabitation should be decidable.

Why normalization doesn't help

Tempting reasoning: System F is strongly normalizing, so every well-typed term has a normal form, so we can search over normal forms, so inhabitation should be decidable.

The gap: normalization says that *given* a well-typed term, it reduces to a normal form. It does **not** bound the search space. A normal form of type A can use sub-derivations at types that have nothing to do with A, and there is no bound (from A alone) on the complexity of those types.

In STLC, every type in a normal proof of A is a subformula of A: the type **controls** the proof. In System F, $\forall$-E lets you instantiate at *any* type B. The proof can detour through types unrelated to the goal.

The search space explodes

In STLC (subformula property): every type appearing in a normal proof of A is a subformula of A. Finitely many subformulas $\Rightarrow$ finitely many candidate terms $\Rightarrow$ enumerate and check.

In System F: the $\forall$-E rule says $t\,[\![B]\!] : A[\alpha := B]$ for *any* type B. When searching for a proof, at each $\forall$-E step we must **guess** B. But there are infinitely many types to choose from, and the right B might be arbitrarily complex and share nothing with the goal type.

So the search tree has **infinite branching** at every $\forall$-E step. There is no computable way to prune it: no finite set of candidate types is guaranteed to suffice.

Theorem (Urzyczyn, 1997)

Type inhabitation in System F is **undecidable**.

Incompleteness and the scorecard

System F encodes enough arithmetic to trigger **Gödel's incompleteness theorem**: some types that "should'' be inhabited have no closed inhabitant.

	STLC	**System T**	**System F**
Normalization			
$\Rightarrow$ consistency			
Decidable type checking			
Decidable inhabitation			✗
Completeness		✗	✗

Why System T inhabitation is decidable

System T types are built from Nat and $\to$. Every such type is inhabited:

- Nat is inhabited by 0.
- $A \to B$ is inhabited by $\lambda x{:}A.\,(\text{inhabitant of } B)$, by induction on B.

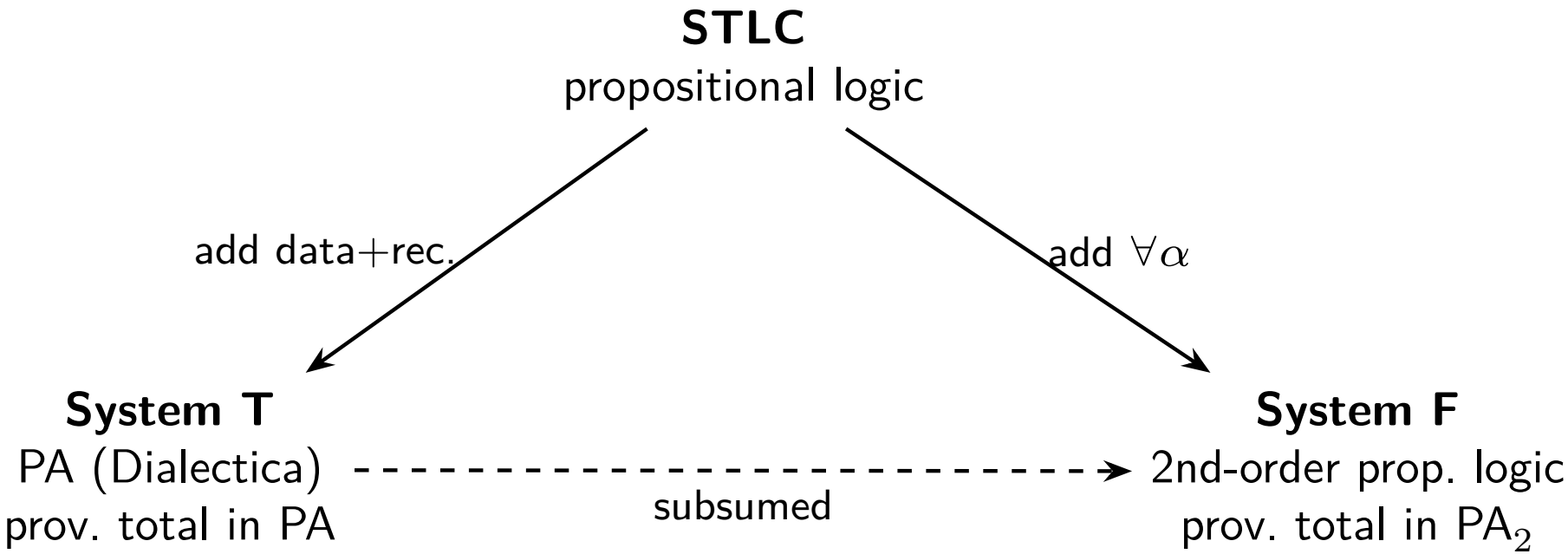
So inhabitation is trivially decidable: the answer is always "yes.''

Caution: this does *not* mean PA is decidable. PA asks whether arithmetic *sentences* are true — a question about $\mathbb{N}$. Inhabitation asks whether a *type* has a closed term — a question about the type system. System T's type language (Nat, $\to$) is far too simple to express arbitrary arithmetic.

Section 6

The Big Picture

Two paths from STLC



Left: add computational primitives, gain power, leave the λ-calculus.

Right: enrich the logic, gain strictly more power, stay in the λ-calculus.

Two senses of "System F corresponds to PA_2''

Be careful: "System F corresponds to PA_2'' means two different things, and conflating them is a common mistake.

Claim	**What it says**
Computational	The functions $\mathbb{N} \to \mathbb{N}$ representable in System F are exactly the provably total functions of PA_2.
Logical	System F types are formulas of 2nd-order *propositional* logic.

The first is about which *functions* you can compute. The second is about which *statements* you can express as types. Don't conflate them.

Example: $\forall n$:Nat. $n + 0 = n$ is **not** a System F type — it quantifies over a term n and uses equality on terms, both of which require dependent types. System F can *compute* $n \mapsto n + 0$, but cannot *state* that this equals the identity.

What System F types cannot say

A proof assistant must *state* specifications as types and *verify* proofs as type-checking. System F can do the verification (decidable type checking), but it cannot state interesting specifications.

In System F, the type of a function says **nothing** about the relationship between input and output *values*:

System F type	**What it guarantees**
List Nat $\to$ List Nat	takes a list, returns a list
Nat $\to$ Nat $\to$ Nat	takes two numbers, returns a number

Both sorting and reversing have the first type. Both addition and multiplication have the second. The types **cannot distinguish** them.

The root cause: types can't see terms

System F's $\forall\alpha$ quantifies over **types**. It cannot quantify over **terms**.

Quantifier	**Ranges over**	**Available?**
$\forall\alpha.\ A$	types (propositions)	System F
$\forall n$:Nat. $P(n)$	terms (individuals)	✗

System F is second-order **propositional** logic. But most specifications are **predicate** logic: they say "for all inputs x, the output satisfies $P(x)$." The word *propositional* is the bottleneck.

The root cause: types can't see terms

System F's $\forall\alpha$ quantifies over **types**. It cannot quantify over **terms**.

Quantifier	**Ranges over**	**Available?**
$\forall\alpha.\ A$	types (propositions)	System F
$\forall n$:Nat. $P(n)$	terms (individuals)	✗

System F is second-order **propositional** logic. But most specifications are **predicate** logic: they say "for all inputs x, the output satisfies $P(x)$." The word *propositional* is the bottleneck.

Try encoding "a list of length n" as a System F type. You cannot: n is a term, and System F types cannot mention terms.

What we want: types that mention values

Imagine writing:

$$\mathsf{sort} : \Pi\,(l : \mathsf{List\ Nat}).\ \Sigma\,(l' : \mathsf{List\ Nat}).\ \mathsf{Sorted}(l') \times \mathsf{Perm}(l, l')$$

$$\mathsf{head} : \Pi\,A.\ \Pi\,(n : \mathsf{Nat}).\ \mathsf{Vec}\ A\ (\mathsf{S}\ n) \to A$$

$$\mathsf{append} : \Pi\,A.\ \Pi\,(m\ n : \mathsf{Nat}).\ \mathsf{Vec}\ A\ m \to \mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ (m + n)$$

In each case the return type depends on *values*: the input list l, the length n, the sum $m + n$. None of this is expressible in System F, because types cannot mention terms.

This is a **logical necessity**, not a convenience: under Curry–Howard, $\Pi\,x{:}A.\ B(x)$ *is* $\forall x{:}A.\ B(x)$. Without it, we have no predicate logic.

Dependent types: the missing axis

Dependent function type: $\Pi\, x{:}A.\ B(x)$ — "given $x : A$, return something of type $B(x)$, where B may mention x."

$$\begin{array}{lcl} \lambda x{:}A.\, t & \longleftrightarrow & \forall\text{-intro (over individuals)} \\ t\ a & \longleftrightarrow & \forall\text{-elim (instantiate with a term)} \end{array}$$

Logic	**Quantification**	**Type system**
Propositional	none	STLC
2nd-order propositional	over propositions	System F
Predicate / higher-order	**over individuals**	**dep. types (CC)**

Combining $\forall\alpha$ (System F) with $\Pi x{:}A.\, B(x)$ (dependent types) gives the **Calculus of Constructions** — the foundation of Coq and Lean.

The lambda cube and what's next

Barendregt: type systems differ by which entities can depend on which.

Dependency	**Meaning**	**System**
terms on terms	functions	STLC ($\lambda\!\to$)
terms on types	polymorphism	System F ($\lambda 2$)
types on types	type operators	F_ω
types on terms	dependent types	LF (λP)

Three independent switches $\Rightarrow 2^3 = 8$ systems forming a cube. All switches on $=$ **Calculus of Constructions** (CC). CC $+$ inductive types $=$ CIC (Coq, Lean).

Next week: type safety for System F; SN of System T (Tait); SN of System F (why Tait breaks, Girard's reducibility candidates).

Stronger logic $\Leftrightarrow$ richer types $\Leftrightarrow$ more terminating programs.